Binary Search

Ulises Tirado Zatarain

Aug, 2021

What is binary search?

- It's a search algorithm to find an element in a sorted set
- The set can be linear structure (vectors) or dynamic (trees)
- There are few variants:
  - Returns whether the element exists (boolean)
  - Returns the position of the element (index, iterator, ...)
  - Returns an index or iterator to the upper/lower bound of target
- Here we will review the implementation looking within a vector

What is binary search?

- It's a search algorithm to find an element in a sorted set
- The set can be linear structure (vectors) or dynamic (trees)
- There are few variants:
  - Returns whether the element exists (boolean)
  - Returns the position of the element (index, iterator, ...)
  - Returns an index or iterator to the upper/lower bound of target
- Here we will review the implementation looking within a vector

What is binary search?

- It's a search algorithm to find an element in a sorted set
- The set can be linear structure (vectors) or dynamic (trees)
- There are few variants:
  - Returns whether the element exists (boolean)
  - Returns the position of the element (index, iterator, ...)
  - Returns an index or iterator to the upper/lower bound of target
- Here we will review the implementation looking within a vector

What is binary search?

- It's a search algorithm to find an element in a sorted set
- The set can be linear structure (vectors) or dynamic (trees)
- There are few variants:
  - Returns whether the element exists (boolean)
  - Returns the position of the element (index, iterator, ...)
  - Returns an index or iterator to the upper/lower bound of target
- Here we will review the implementation looking within a vector

What is binary search?

- It's a search algorithm to find an element in a sorted set
- The set can be linear structure (vectors) or dynamic (trees)
- There are few variants:
  - Returns whether the element exists (boolean)
  - Returns the position of the element (index, iterator, ...)
  - Returns an index or iterator to the upper/lower bound of target
- Here we will review the implementation looking within a vector

What is binary search?

- It's a search algorithm to find an element in a sorted set
- The set can be linear structure (vectors) or dynamic (trees)
- There are few variants:
  - Returns whether the element exists (boolean)
  - Returns the position of the element (index, iterator, ...)
  - Returns an index or iterator to the upper/lower bound of target
- Here we will review the implementation looking within a vector

What is binary search?

- It's a search algorithm to find an element in a sorted set
- The set can be linear structure (vectors) or dynamic (trees)
- There are few variants:
  - Returns whether the element exists (boolean)
  - Returns the position of the element (index, iterator, ...)
  - Returns an index or iterator to the upper/lower bound of target
- Here we will review the implementation looking within a vector

Algorithm details

- The idea consists in make a partition in the set
- If the element middle of the partition is equal to the target value, we found it!
- Otherwise, discard one of the sides depending if the element is bigger or smaller than the one in the middle
- The implementation can be iterative or recursive
- It's time complexity is $\mathcal{O}(\log n)$

Algorithm details

- The idea consists in make a partition in the set
- If the element middle of the partition is equal to the target value, we found it!
- Otherwise, discard one of the sides depending if the element is bigger or smaller than the one in the middle
- The implementation can be iterative or recursive
- It's time complexity is $\mathcal{O}(\log n)$

Algorithm details

- The idea consists in make a partition in the set
- If the element middle of the partition is equal to the target value, we found it!
- Otherwise, discard one of the sides depending if the element is bigger or smaller than the one in the middle
- The implementation can be iterative or recursive
- It's time complexity is $\mathcal{O}(\log n)$

Algorithm details

- The idea consists in make a partition in the set
- If the element middle of the partition is equal to the target value, we found it!
- Otherwise, discard one of the sides depending if the element is bigger or smaller than the one in the middle
- The implementation can be iterative or recursive
- It's time complexity is $\mathcal{O}(\log n)$

Algorithm details

- The idea consists in make a partition in the set
- If the element middle of the partition is equal to the target value, we found it!
- Otherwise, discard one of the sides depending if the element is bigger or smaller than the one in the middle
- The implementation can be iterative or recursive
- It's time complexity is $\mathcal{O}(\log n)$

Implementation (recursive)

```
bool binary_search(
    const std::vector<int>& elements,
    int target, int begin, int end
) {
    if (begin > end) return false;
    int mid = (begin + end) /2;
    if (elements[mid] == target) return true;
    if (target > elements[mid]) {
        return binary_search(
            elements, target, mid + 1, end
        );
    }
    return binary_search(
        elements, target, begin, mid - 1
    );
}
```

Implementation (recursive)

Overload the function to reduce number of parameters:

```
bool binary_search(
    const std::vector<int>& elements,
    int target
) {
    return binary_search(
        elements, target, 0,
        elements.size() - 1
    );
}
```

Implementation (iterative)

```
bool binary_search(
    const std::vector<int>& elements,
    int target
) {
    int begin = 0, end = elements.size() - 1;
    while (begin < end) {
        int mid = (begin + end) /2;
        if (elements[mid] == target) return true;
        if (target > elements[mid]) {
            begin = mid + 1;
        } else {
            end = mid - 1;
        }
    }
    return false;
}
```

STL template and iterator-based implementations

Checks if an element equivalent to value appears within the range [first, last).

```
template<class Iterator, class Value>
bool std::binary_search(
    Iterator first, Iterator last,
    const Value& target
);

template<class Iterator, class Value,
    class Comparator>
bool std::binary_search(
    Iterator first, Iterator last,
    const Value& target, Comparator compare
);
```

STL even more interesting variations: std::lower_bound

Returns an iterator pointing to the first element in the range [first, last) that is not less than (i.e. greater or equal to) value, or last if no such element is found.

```
template<class Iterator, class Value>
Iterator std::lower_bound(
    Iterator first, Iterator last,
    const Value& target
);

template<class Iterator, class Value,
    class Comparator>
Iterator std::lower_bound(
    Iterator first, Iterator last,
    const Value& target, Comparator compare
);
```

STL even more interesting variations: std::upper_bound

Returns an iterator pointing to the first element in the range [first, last) that is greater than value, or last if no such element is found.

```
template<class Iterator, class Value>
Iterator std::upper_bound(
    Iterator first, Iterator last,
    const Value& target
);

template<class Iterator, class Value,
    class Comparator>
Iterator std::upper_bound(
    Iterator first, Iterator last,
    const Value& target, Comparator compare
);
```

References

- Wikipedia: Binary Search Algorithm
- C++ Binary Search
- C++ Lower Bound
- C++ Upper Bound
- BBC: Bitesize